

pqcoin: A Post-Quantum Cryptocurrency

Version 0.1.0

Abstract

pqcoin is a proof-of-work cryptocurrency designed to provide security against both classical and quantum computers. By utilizing NIST-standardized post-quantum cryptographic primitives—specifically ML-DSA-87 (FIPS 204) for digital signatures and SHA3-512 (FIPS 202) for hashing—pqcoin offers a quantum-resistant alternative to existing cryptocurrencies that rely on elliptic curve cryptography. The design follows Bitcoin’s proven UTXO model while making deliberate simplifications: removing the scripting language in favor of two well-defined output types (P2PKH and M-of-N multisig), and modernizing the cryptographic foundation for the post-quantum era.

Contents

1 Introduction

1.1 Motivation

Current cryptocurrencies like Bitcoin rely on the Elliptic Curve Digital Signature Algorithm (ECDSA) for transaction authorization. While secure against classical computers, ECDSA is vulnerable to Shor’s algorithm running on a sufficiently powerful quantum computer. As quantum computing advances, the cryptocurrency ecosystem faces an existential threat to its security model.

pqcoin addresses this threat by building on cryptographic primitives that are believed to be secure against both classical and quantum attacks, as standardized by NIST in their Post-Quantum Cryptography project.

1.2 Design Goals

1. **Quantum Resistance:** Use NIST-standardized post-quantum cryptographic algorithms
2. **Simplicity:** Remove unnecessary complexity while maintaining Bitcoin’s proven security model
3. **Correctness:** Prioritize security and correctness over feature richness
4. **Compatibility:** Follow familiar patterns from Bitcoin where appropriate

2 Cryptographic Primitives

2.1 Hash Function: SHA3-512 (FIPS 202)

All hashing operations in pqcoin use SHA3-512, providing:

- **Output size:** 512 bits (64 bytes)
- **Security level:** 256-bit classical security
- **Quantum security:** $\sim$ 256-bit security against Grover’s algorithm (effectively 128-bit due to birthday attacks)

SHA3-512 is used for:

- Transaction ID computation
- Block header hashing (proof-of-work)
- Address derivation (hash of public key)
- Merkle tree construction

2.2 Digital Signatures: ML-DSA-87 (FIPS 204)

Transaction authorization uses ML-DSA-87 (formerly Dilithium5), a lattice-based digital signature scheme. Table ?? summarizes the key sizes.

ML-DSA-87 provides NIST Level 5 security ($\sim$ 256-bit classical, $\sim$ 128-bit quantum security). This is the highest security level in the ML-DSA family, providing maximum protection against future advances in quantum computing.

Table 1: ML-DSA-87 Key and Signature Sizes

Component	Size (bytes)
Public Key	2,592
Secret Key	4,896
Signature	4,627

2.3 Signature Construction

Signatures are computed over the SHA3-512 hash of the transaction’s signing data:

```
message = SHA3-512(signing_data(tx, input_index))
signature = ML-DSA-87.Sign(secret_key, message)
```

This approach:

1. Provides a fixed-size input regardless of transaction size
2. Matches Bitcoin’s practice of signing transaction hashes
3. Is compatible with ML-DSA-87’s design for signing arbitrary messages

3 Transaction Model

3.1 UTXO Model

pqcoin uses the Unspent Transaction Output (UTXO) model, identical in concept to Bitcoin:

- Transactions consume existing outputs (inputs) and create new outputs
- Each output can only be spent once
- The sum of input values must equal or exceed output values
- The difference (inputs – outputs) is the transaction fee

3.2 Output Types

Unlike Bitcoin’s scripting system, pqcoin supports exactly two output types:

3.2.1 Pay-to-Public-Key-Hash (P2PKH)

The standard single-signature output. Funds are locked to the SHA3-512 hash of a public key. To spend:

1. Provide the public key that hashes to the address
2. Provide a valid signature over the transaction

3.2.2 M-of-N Multisig

Multi-signature outputs requiring M signatures from N possible signers. Limited to a maximum of 15 keys due to post-quantum signature sizes.

3.3 Transaction Structure

```
Transaction {
    version: u32,           // Currently 1
    inputs: Vec<TxInput>,   // References to outputs being spent
    outputs: Vec<TxOutput>, // New outputs being created
}

TxInput {
    outpoint: OutPoint,     // (txid, index) reference
    witness: Witness,       // Proof of authorization
}

TxOutput {
    amount: u64,            // Value in quanta
    condition: LockingCondition, // P2PKH or Multisig
}
```

3.4 Dust Limit

To prevent UTXO set bloat, outputs must exceed a dynamic dust limit based on the cost to spend them:

$$\text{dust_limit} = \frac{\text{estimated_spend_size} \times \text{DUST_FEE_RATE}}{1000} \quad (1)$$

Where $\text{DUST_FEE_RATE} = 100$ quanta per KB. This ensures outputs are economically rational to spend, accounting for the larger post-quantum signatures.

4 Block Structure

4.1 Block Header

The block header is 176 bytes, structured as shown in Table ??.

Table 2: Block Header Structure

Field	Size	Description
version	4 bytes	Block format version
prev_hash	64 bytes	SHA3-512 hash of previous header
merkle_root	64 bytes	Root of transaction merkle tree
timestamp	8 bytes	Unix timestamp
difficulty_bits	4 bytes	Compact difficulty target
nonce	32 bytes	Proof-of-work nonce

4.2 256-bit Nonce

ppcoin uses a 32-byte (256-bit) nonce, providing 2^{256} possible values. This design:

- Eliminates nonce exhaustion concerns
- Removes the need for Bitcoin-style extraNonce mechanisms
- Simplifies mining implementation

4.3 Merkle Root

The merkle root commits to all transactions using a binary hash tree:

- Leaves are transaction IDs (SHA3-512 hashes)
- Internal nodes are SHA3-512 hashes of concatenated children
- Odd levels duplicate the last node

4.4 Block Limits

Table 3: Block Limits

Parameter	Value
Max transactions	1,000 (excluding coinbase)
Max block size	16 MB
Max inputs per tx	10,000
Max outputs per tx	10,000
Max multisig keys	15

The 16 MB block size (vs Bitcoin’s effective ~ 4 MB) accommodates the larger post-quantum signatures.

5 Consensus Mechanism

5.1 Proof of Work

Valid blocks must have a header hash less than or equal to the target:

$$\text{SHA3-512}(\text{block_header}) \leq \text{target} \quad (2)$$

The target is derived from `difficulty_bits` using the formula:

$$\text{target} = \text{coefficient} \times 2^{8 \times (\text{exponent} - 3)} \quad (3)$$

Where `difficulty_bits` = (`exponent` $\ll$ 24) | `coefficient`.

5.2 Difficulty Adjustment

Difficulty adjusts every 2,016 blocks to maintain a 10-minute average block time:

1. Calculate actual time for the previous 2,016 blocks
2. Clamp adjustment to $4\times$ in either direction
3. Scale the target: $\text{new_target} = \text{old_target} \times \frac{\text{actual_time}}{\text{target_time}}$

A minimum difficulty floor (`MIN_DIFFICULTY_BITS` = 0x41ffff) prevents difficulty from dropping below a safe threshold.

5.3 Chain Selection

The valid chain with the most cumulative proof-of-work is the canonical chain. Reorganizations are limited to 100 blocks to prevent long-range attacks.

5.4 Timestamp Rules

Block timestamps must:

1. Be strictly greater than the Median-Time-Past (median of previous 11 blocks)
2. Not exceed current time + 2 hours

This prevents timestamp manipulation attacks near difficulty adjustment boundaries.

6 Monetary Policy

6.1 Block Reward

Table 4: Monetary Parameters

Parameter	Value
Initial reward	50,000,000 quanta (50 coins)
Halving interval	210,000 blocks
Smallest unit	1 quantum

6.2 Supply Schedule

Following Bitcoin's model:

- Total supply asymptotically approaches 21 million coins
- Reward halves every ~ 4 years (at 10-minute blocks)
- After 64 halvings, reward becomes 0

6.3 Coinbase Maturity

Coinbase outputs cannot be spent until 100 blocks have been mined on top of them. This prevents issues from chain reorganizations affecting miner rewards.

7 Network Protocol

7.1 Message Format

Messages use a length-prefixed framing format:

```
[Magic: 4 bytes "PQCN"]  
[Command: 12 bytes]  
[Length: 4 bytes]  
[Checksum: 4 bytes]  
[Payload: variable]
```

The checksum is the first 4 bytes of SHA3-512(payload).

7.2 Message Types

7.3 Peer Management

Subnet limits help prevent eclipse attacks.

Table 5: Network Message Types

Message	Purpose
version/verack	Handshake and capability exchange
ping/pong	Keepalive
inv/getdata	Inventory announcement and request
getheaders/headers	Header synchronization
block/tx	Block and transaction relay
addr/addrv2	Peer discovery

Table 6: Network Parameters

Parameter	Value
Default port	8333
Max connections	125 (8 outbound)
Max per IP	3
Max per /16 subnet	2
Ban duration	24 hours
Ping interval	60 seconds

8 Wallet

8.1 Key Storage

Wallet files use encrypted JSON format:

- Secret keys encrypted with AES-256-GCM
- Key derivation via Argon2 from user password
- Memory is zeroized after use

8.2 Address Format

Addresses are the full 64-byte SHA3-512 hash of the public key, displayed as 128 hexadecimal characters.

9 Security Considerations

9.1 Quantum Resistance

ppcoin’s security against quantum computers rests on:

1. **ML-DSA-87**: Based on the Module-LWE problem, believed to be hard for quantum computers
2. **SHA3-512**: Grover’s algorithm provides only quadratic speedup, leaving 256-bit security

9.2 Classical Security

Against classical computers:

- 256-bit hash security (SHA3-512)
- 256-bit signature security (ML-DSA-87 at NIST Level 5)

9.3 Known Limitations

1. **Signature size:** ML-DSA-87 signatures (4,627 bytes) are $\sim 72\times$ larger than ECDSA (64 bytes), affecting transaction throughput
2. **No scripting:** Complex smart contracts are not supported
3. **No chained transactions:** Mempool does not support spending unconfirmed outputs

10 Comparison with Bitcoin

Table 7: pqcoin vs Bitcoin

Feature	pqcoin	Bitcoin
Hash function	SHA3-512 (64 bytes)	SHA-256 (32 bytes)
Signature scheme	ML-DSA-87	ECDSA (secp256k1)
Signature size	4,627 bytes	64 bytes
Public key size	2,592 bytes	33 bytes
Quantum resistant	Yes	No
Scripting	No (P2PKH + Multisig)	Yes (Script)
Nonce size	256 bits	32 bits
Block size limit	16 MB	~ 4 MB

11 Future Work

1. **Light clients:** Bloom filtering for SPV wallets
2. **Checkpoints:** Hardcoded block hashes for faster initial sync
3. **Assume-valid:** Skip signature verification for deeply-buried blocks
4. **Hardware wallet support:** Integration with post-quantum capable hardware

12 Conclusion

pqcoin provides a quantum-resistant cryptocurrency built on NIST-standardized cryptographic primitives. By simplifying Bitcoin’s design while upgrading its cryptographic foundation, pqcoin offers a secure platform for value transfer in the post-quantum era.

The deliberate constraints—no scripting, limited output types, no chained transactions—prioritize security and correctness over feature richness, making the codebase more auditable and the security model more analyzable.

References

1. NIST FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions
2. NIST FIPS 204: Module-Lattice-Based Digital Signature Standard
3. Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System
4. NIST Post-Quantum Cryptography Standardization Process

A Protocol Constants

```
// Protocol
MAX_BLOCK_TXS = 1,000
MAX_TX_INPUTS = 10,000
MAX_TX_OUTPUTS = 10,000
MAX_MULTISIG_KEYS = 15
MAX_BLOCK_SIZE = 16 MB
COINBASE_MATURITY = 100 blocks
MAX_REORG_DEPTH = 100 blocks
MAX_FUTURE_BLOCK_TIME = 2 hours

// Mining
INITIAL_REWARD = 50,000,000 quanta
DIFFICULTY_INTERVAL = 2,016 blocks
TARGET_BLOCK_TIME = 600 seconds
HALVING_INTERVAL = 210,000 blocks
MIN_DIFFICULTY_BITS = 0x41ffffff

// Mempool
MIN_RELAY_FEE = 1,000 quanta/byte
DUST_FEE_RATE = 100 quanta/KB
TX_EXPIRY_BLOCKS = 432 blocks (~72 hours)
MAX_MEMPOOL_SIZE = 5,000 transactions

// Network
DEFAULT_PORT = 8333
MAX_PEERS = 125
MAX_OUTBOUND = 8
BAN_DURATION = 24 hours
PING_INTERVAL = 60 seconds
```

B Cryptographic Key Sizes

Table 8: Cryptographic Component Sizes

Algorithm	Component	Size (bytes)
SHA3-512	Hash output	64
ML-DSA-87	Public key	2,592
ML-DSA-87	Secret key	4,896
ML-DSA-87	Signature	4,627
AES-256-GCM	Key	32
AES-256-GCM	Nonce	12
Argon2	Salt	16